

PROJEKT SEMESTRALNY

System rezerwacji usług

Kompletny opis projektu, wymagania i kryteria oceniania

PRZEDMIOT	Tworzenie stron i aplikacji internetowych
KLASA	5 technikum informatycznego
CZAS REALIZACJI	wrzesień-listopad 2026
ORGANIZACJA	projekt w parach, jedno zadanie grupowe w Moodle
REPOZYTORIUM	jeden wspólny link GitHub używany przez cały projekt

Dokument przeznaczony do publikacji jako opis jednego zadania grupowego w Moodle.

Spis treści

1. Informacje najważniejsze	4
1.1. Tematyka	4
1.2. Zgłoszenie w Moodle	4
1.3. Technologie	4
2. Organizacja pracy i repozytorium GitHub	4
2.1. Wymagania dotyczące historii pracy	4
2.2. Minimalna organizacja repozytorium	5
3. Role użytkowników	5
3.1. Klient	5
3.2. Pracownik	5
3.3. Administrator	5
4. Relacyjna baza danych	5
4.1. Wymagane elementy projektu bazy	6
5. Rejestracja, logowanie, sesje i role	6
5.1. Rejestracja	6
5.2. Logowanie i kontrola dostępu	6
6. Oferta usług	6
7. Proces rezerwacji i dostępność	6
7.1. Reguła nakładania przedziałów	7
8. Statusy i panele użytkowników	7
8.1. Statusy rezerwacji	7
8.2. Panel klienta	7
8.3. Panel pracownika	7
8.4. Panel administratora	8
9. Bezpieczeństwo i jakość aplikacji	8
10. Interfejs i responsywność	8
11. Funkcje dodatkowe	8
12. Harmonogram i pięć punktów kontrolnych	9
13. Szczegółowe kryteria oceniania	9
Etap 1. Projekt systemu, GitHub i baza danych	10

Etap 2. Użytkownicy, rejestracja i logowanie	12
Etap 3. Panel administracyjny	14
Etap 4. Część klienta i system rezerwacji	16
Etap 5. Gotowy projekt i prezentacja	18
14. Zasady oceny indywidualnej i obrony	20
14.1. Pytania i zadania podczas sprawdzania	20
14.2. Prezentacja końcowa	20
15. Frameworki, biblioteki, szablony i AI	20
15.1. Frameworki i biblioteki	20
15.2. Gotowe szablony	20
15.3. Kod przygotowany z pomocą AI	20
15.4. Funkcje dodatkowe	20
16. Lista kontrolna przed prezentacją	20
17. Najważniejsza zasada projektu	21

1. Informacje najważniejsze

Waszym zadaniem jest zaprojektowanie i wykonanie kompletnej aplikacji internetowej umożliwiającej klientom rezerwowanie terminów usług. Projekt realizujecie w parach przez wrzesień, październik i listopad 2026 r.

W Moodle znajduje się jedno zadanie grupowe. Każda para przekazuje w nim jeden raz nazwę projektu, autorów, tematykę oraz link do wspólnego repozytorium GitHub. Ten sam link jest używany podczas wszystkich pięciu punktów kontrolnych. Nie tworzycie osobnych repozytoriów ani osobnych zgłoszeń dla kolejnych etapów.

1.1. Tematyka

System może być przeznaczony m.in. dla salonu fryzjerskiego, gabinetu kosmetycznego, warsztatu samochodowego, serwisu komputerowego, gabinetu fizjoterapii, szkoły językowej, studia fotograficznego, firmy sprzątającej lub innej firmy świadczącej usługi. Możecie zaproponować własną tematykę. Tematyka wpływa na wygląd oraz ofertę, ale podstawowe wymagania są wspólne dla wszystkich zespołów.

1.2. Zgłoszenie w Moodle

Jedna osoba z pary uzupełnia w zadaniu grupowym:

Nazwa projektu	
Autorzy	
Tematyka systemu	
Repozytorium GitHub	https://github.com/

1.3. Technologie

- PHP
- HTML5
- CSS3
- JavaScript
- MySQL lub MariaDB
- Git i GitHub

Frameworki są opcjonalne. Projekt może powstać w czystym PHP albo np. w Laravelu. Dozwolone są biblioteki i komponenty takie jak Bootstrap, Tailwind CSS, jQuery lub FullCalendar, zgodnie z zasadami opisanymi w dalszej części dokumentu.

2. Organizacja pracy i repozytorium GitHub

Każdy zespół zakłada jedno wspólne repozytorium. Obaj uczniowie muszą mieć dostęp do repozytorium i aktywnie uczestniczyć w jego tworzeniu.

2.1. Wymagania dotyczące historii pracy

- regularne wykonywanie commitów przez obie osoby przez cały okres realizacji projektu,
- sensowne i konkretne opisy commitów,
- korzystanie z osobnych gałęzi przy tworzeniu większych funkcjonalności,
- scalanie zmian z główną gałęzią oraz rozwiązywanie konfliktów Git,
- prowadzenie aktualnego pliku README.md,
- umieszczanie kodu obu osób w repozytorium.

Nie jest dopuszczalna sytuacja, w której jedna osoba tworzy cały projekt, a druga jedynie przesyła jej pliki. Sama liczba commitów nie decyduje o ocenie; liczą się ich regularność, sens i rzeczywisty zakres zmian.

2.2. Minimalna organizacja repozytorium

```
README.md
/database/database.sql
/public lub główny katalog aplikacji
/css
/js
/images lub /assets
```

README.md powinien zawierać: nazwę projektu, imiona i nazwiska autorów, opis systemu, zastosowane technologie, instrukcję uruchomienia, testowe dane logowania do kont administratora i pracownika oraz opis funkcji dodatkowych.

3. Role użytkowników

3.1. Klient

- zakłada konto, loguje się i wylogowuje,
- przegląda ofertę i szczegóły usług,
- wybiera usługę, pracownika i dostępny termin,
- tworzy rezerwację,
- przegląda wyłącznie własne rezerwacje,
- anuluje własną przyszłą rezerwację,
- edytuje podstawowe dane profilu.

3.2. Pracownik

- loguje się do swojego panelu,
- widzi wizyty na dziś, przyszłe wizyty oraz historię,
- otwiera szczegóły własnych rezerwacji,
- zmienia status obsługiwanej wizyty,
- widzi potrzebne informacje o kliencie i usłudze.

3.3. Administrator

- zarządza użytkownikami, pracownikami, kategoriami i usługami,
- przypisuje usługi pracownikom i określa ich dostępność,
- przegląda i filtruje wszystkie rezerwacje,
- zmienia statusy i anuluje rezerwacje,
- przegląda podstawowe statystyki systemu.

4. Relacyjna baza danych

Baza może mieć inną poprawną strukturę niż przykładowa poniżej, ale musi zachowywać wymagane relacje, integralność danych oraz możliwość realizacji wszystkich funkcji systemu.

TABELA	PRZYKŁADOWE POLA / ROLA
users	id, name, surname, email, password, phone, role, active, created_at
service_categories	id, name, description
services	id, category_id, name, description, duration, price, active
employees	id, user_id, description, active (lub odpowiednie dane w users)

TABELA	PRZYKŁADOWE POLA / ROLA
employee_services	employee_id, service_id - relacja N:M
employee_availability	id, employee_id, day_of_week lub date, start_time, end_time
reservations	id, user_id, employee_id, service_id, reservation_date, start_time, end_time, status, comment, created_at

4.1. Wymagane elementy projektu bazy

- klucze główne i klucze obce,
- relacje 1:N oraz przynajmniej jedna relacja N:M,
- właściwe typy danych,
- ograniczenia integralności danych,
- diagram ERD lub inny czytelny schemat,
- plik database.sql pozwalający odtworzyć bazę, relacje i dane testowe.

5. Rejestracja, logowanie, sesje i role

5.1. Rejestracja

Formularz rejestracji zawiera co najmniej: imię, nazwisko, adres e-mail, telefon, hasło i powtórzenie hasła. Serwer sprawdza wymagane pola, format adresu e-mail, unikalność e-maila oraz zgodność haseł.

Hasła nie mogą być przechowywane jawnie. W PHP należy wykorzystać `password_hash()` i `password_verify()` albo równoważny bezpieczny mechanizm frameworka.

5.2. Logowanie i kontrola dostępu

- logowanie za pomocą e-maila i hasła oraz możliwość wylogowania,
- utrzymywanie stanu zalogowania w sesji,
- rozpoznawanie ról: klient, pracownik, administrator,
- przekierowanie do właściwego panelu po zalogowaniu,
- ochrona wszystkich podstron i operacji po stronie serwera.

Samo ukrycie linku w interfejsie nie jest zabezpieczeniem. Klient nie może wejść do panelu administratora lub pracownika przez ręczne wpisanie adresu URL.

6. Oferta usług

- publiczna lista kategorii i usług,
- szczegóły usługi: nazwa, opis, kategoria, cena, czas trwania i informacja o dostępności,
- wskazane filtrowanie oferty według kategorii,
- wyświetlanie wyłącznie aktywnych usług przeznaczonych dla klientów.

7. Proces rezerwacji i dostępność

Proces obowiązkowo przebiega w kolejności:

usługa -> pracownik -> data -> godzina -> potwierdzenie

1. Po wybraniu usługi system pokazuje wyłącznie pracowników, którzy ją wykonują.
2. Po wybraniu pracownika i daty system wyznacza dostępne godziny.
3. Dostępność uwzględnia godziny pracy pracownika i czas trwania wybranej usługi.

4. System nie pozwala rezerwować w przeszłości, poza godzinami pracy ani w terminie kolidującym z inną aktywną rezerwacją.
5. Konflikt jest ponownie sprawdzany po stronie serwera bezpośrednio przed zapisem.
6. Po poprawnej walidacji użytkownik potwierdza rezerwację, a dane zostają zapisane w bazie.

7.1. Reguła nakładania przedziałów

Dwie wizyty kolidują, gdy początek nowej wizyty przypada przed końcem istniejącej, a koniec nowej wizyty przypada po początku istniejącej. Sprawdzenie musi obejmować wszystkie rodzaje częściowego i całkowitego nakładania.

Istniejąca rezerwacja: 10:00-11:00

System musi zablokować: 9:30-10:30, 10:00-11:00, 10:15-10:45, 10:30-11:30 oraz 9:30-11:30.

Dopuszczalne: terminy kończące się dokładnie o 10:00 lub zaczynające dokładnie o 11:00, jeżeli spełniają pozostałe warunki dostępności.

Ukrycie niedostępnej godziny w JavaScript nie jest wystarczające. Użytkownik nie może ominąć kontroli, zmieniając dane formularza lub parametry żądania.

8. Statusy i panele użytkowników

8.1. Statusy rezerwacji

- oczekująca,
- potwierdzona,
- zrealizowana,
- anulowana.

Administrator lub właściwy pracownik może zmieniać status. Klient może anulować wyłącznie własną przyszłą rezerwację. Zaleca się zmianę statusu zamiast fizycznego usuwania rezerwacji.

8.2. Panel klienta

- sekcja Moje konto i edycja profilu,
- podział rezerwacji na przyszłe oraz zakończone/anulowane,
- dla każdej pozycji: usługa, pracownik, data, godzina, cena i status,
- anulowanie własnej przyszłej rezerwacji.

8.3. Panel pracownika

- wizyty na dzisiaj, przyszłe wizyty i historia,
- szczegóły rezerwacji i zmiana statusu,
- brak dostępu do wizyt innego pracownika, o ile nie istnieje jawnie uzasadnione uprawnienie.

8.4. Panel administratora

MODUŁ	MINIMALNY ZAKRES
Dashboard	liczba klientów, pracowników, usług, wszystkich rezerwacji i rezerwacji na dziś
Kategorie	dodawanie, edycja, usuwanie lub dezaktywowanie
Usługi	CRUD, kategoria, cena, czas trwania, aktywność
Pracownicy	dane, aktywność, przypisane usługi i dostępność
Użytkownicy	przeglądanie i zarządzanie zgodne z przyjętymi uprawnieniami
Rezerwacje	lista, szczegóły, status, anulowanie; filtrowanie według daty, statusu i pracownika

9. Bezpieczeństwo i jakość aplikacji

- hashowanie i bezpieczna weryfikacja haseł,
- prepared statements w PDO/MySQLi albo bezpieczny mechanizm ORM,
- walidacja danych po stronie serwera, niezależnie od walidacji w JavaScript,
- zabezpieczanie danych wyświetlanych w HTML przed XSS,
- kontrola sesji, roli i uprawnień dla każdej chronionej strony oraz operacji,
- kontrola własności danych: klient nie może zobaczyć ani zmienić cudzej rezerwacji przez modyfikację identyfikatora,
- komunikaty błędów nie mogą ujawniać danych technicznych aplikacji lub bazy,
- repozytorium nie może zawierać haseł, kluczy API ani innych sekretów.

Przykład niedopuszczalnej sytuacji: klient zmienia `reservation.php?id=152` na `id=153` i uzyskuje dostęp do cudzej rezerwacji. System musi za każdym razem sprawdzić właściciela danych.

10. Interfejs i responsywność

- czytelna nawigacja i spójna kolorystyka,
- estetyczne formularze, tabele i komunikaty,
- poprawne działanie na komputerze oraz możliwość wygodnej obsługi na telefonie,
- czytelna informacja o błędach i udanych operacjach.

11. Funkcje dodatkowe

Funkcje dodatkowe mogą podnieść ocenę tylko wtedy, gdy działają, są zintegrowane z projektem, mają uzasadnienie i autorzy potrafią je wyjaśnić. Nie zastępują brakujących funkcji obowiązkowych.

- graficzny kalendarz lub FullCalendar,
- AJAX i dynamiczne pobieranie terminów,
- e-maile i przypomnienia,
- resetowanie hasła lub potwierdzanie e-maila,
- różne godziny pracy, urlopy, dni wolne i przerwy,
- historia zmian statusów lub log operacji administratora,
- wykresy, eksport danych lub generowanie PDF,
- wyszukiwanie i paginacja,
- REST API, migracje, testy automatyczne lub rozbudowana architektura.

12. Harmonogram i pięć punktów kontrolnych

Każdy punkt kontrolny kończy się oceną w skali 1-6. Nauczyciel sprawdza aktualny stan tego samego repozytorium zgłoszonego w jednym zadaniu Moodle.

ETAP	TERMIN	ZAKRES	STAN OCZEKIWANY
1	25.09.2026	Projekt, GitHub i baza	repozytorium, struktura, README, ERD, database.sql, relacje, dane testowe
2	16.10.2026	Użytkownicy i logowanie	rejestracja, logowanie, sesje, role, ochrona stron, profil
3	06.11.2026	Panel administracyjny	dashboard, CRUD, pracownicy, usługi, dostępność, rezerwacje
4	20.11.2026	Klient i rezerwacje	oferta, proces rezerwacji, konflikty, panele klienta i pracownika
5	27.11.2026	Kompletny projekt i prezentacja	testy, bezpieczeństwo, responsywność, dokumentacja, obrona kodu

13. Szczegółowe kryteria oceniania

Wymagania mają charakter narastający. Aby otrzymać ocenę 5, należy spełnić wymagania ocen 2, 3, 4 i 5. Przy każdym etapie oceniane są: poprawność działania, jakość kodu, bezpieczeństwo, zgodność z wymaganiami, historia Git, wkład obu osób i umiejętność wyjaśnienia rozwiązań. Samo istnienie kodu nie oznacza wykonania funkcji - musi ona działać zgodnie z wymaganiami.

Etap 1. Projekt systemu, GitHub i baza danych

TERMIN	25 września 2026 r.
OCENIE PODLEGA	organizacja projektu, repozytorium, baza danych, relacje i przygotowanie do dalszej pracy

Ocena 1 - niedostateczny

- nie utworzono repozytorium GitHub,
- nie przygotowano bazy danych,
- baza składa się jedynie z pojedynczych, niepowiązanych tabel,
- projekt nie pozwala na kontynuowanie pracy,
- materiały nie pozwalają stwierdzić samodzielnego wykonania etapu.

Ocena 2 - dopuszczający

- utworzone repozytorium GitHub,
- rozpoczęta struktura aplikacji,
- utworzona baza danych,
- tabele przechowujące przynajmniej użytkowników, usługi i rezerwacje,
- zastosowanie kluczy głównych,
- projekt bazy lub plik SQL w repozytorium.

Dopuszczalne są błędy w relacjach i niepełny projekt bazy.

Ocena 3 - dostateczny

- wszystko na ocenę 2,
- baza zawiera wszystkie najważniejsze elementy systemu,
- tabele użytkowników, usług, kategorii, pracowników i rezerwacji,
- klucze obce i poprawne relacje 1:N,
- powiązanie rezerwacji z klientem, pracownikiem i usługą,
- w większości właściwe typy danych,
- README.md,
- własne commity obu członków zespołu.

Ocena 4 - dobry

- wszystko na ocenę 3,
- mechanizm określania dostępności pracowników,
- relacja N:M między pracownikami i usługami,
- poprawne klucze obce i status rezerwacji,
- brak oczywistego, niepotrzebnego powielania danych,
- diagram ERD lub inny czytelny schemat bazy,
- plik SQL odtwarzający strukturę bazy,
- uporządkowane katalogi i sensowne opisy commitów.

Ocena 5 - bardzo dobry

- wszystko na ocenę 4,
- poprawnie znormalizowana baza,
- odpowiednie ograniczenia integralności,
- UNIQUE tam, gdzie jest uzasadnione, np. dla e-maila,

- indeksy w uzasadnionych miejscach,
- dane przykładowe do testów,
- kompletny database.sql z relacjami,
- README z opisem, autorami, technologiami i instrukcją uruchomienia,
- regularna historia pracy obu osób,
- gałęzie lub inny uporządkowany sposób pracy przy większych elementach.

Ocena 6 - celujący

- wszystko na ocenę 5,
- przynajmniej kilka rzeczywiście użytecznych rozszerzeń, np. historia statusów, osobny system urlopów lub dni wolnych, zaawansowany model godzin pracy, log administratora, migracje, przemyślany system uprawnień, dodatkowe ograniczenia integralności albo szczególnie dobra architektura bazy.

Rozwiązania dodatkowe muszą mieć rzeczywiste zastosowanie w projekcie.

Etap 2. Użytkownicy, rejestracja i logowanie

TERMIN	16 października 2026 r.
OCENIE PODLEGA	rejestracja, logowanie, sesje, role, profil i bezpieczeństwo uwierzytelniania

Ocena 1 - niedostateczny

- nie można poprawnie zarejestrować lub zalogować użytkownika,
- mechanizm logowania praktycznie nie działa,
- hasła są przechowywane jawnie,
- brak mechanizmu sesji,
- poważne błędy uniemożliwiają korzystanie z aplikacji.

Ocena 2 - dopuszczający

- formularz rejestracji i zapis użytkownika w bazie,
- formularz logowania i możliwość poprawnego zalogowania,
- podstawowe użycie sesji,
- możliwość wylogowania,
- hashowanie haseł.

Dopuszczalne są braki w walidacji i systemie uprawnień.

Ocena 3 - dostateczny

- wszystko na ocenę 2,
- `password_hash()` i `password_verify()` lub równoważny bezpieczny mechanizm,
- sprawdzanie pól wymaganych, formatu e-maila i zgodności haseł,
- zakaz duplikowania adresu e-mail,
- poprawne dane zalogowanego użytkownika w sesji,
- ochrona podstawowych stron wymagających logowania,
- komunikaty błędów logowania i rejestracji.

Ocena 4 - dobry

- wszystko na ocenę 3,
- trzy role: klient, pracownik i administrator,
- odpowiednie uprawnienia każdej roli,
- klient nie wchodzi do panelu administratora ani pracownika przez URL,
- kontrola uprawnień po stronie PHP,
- profil użytkownika i edycja podstawowych danych,
- walidacja serwerowa formularzy.

Ocena 5 - bardzo dobry

- wszystko na ocenę 4,
- prepared statements lub bezpieczny ORM,
- kontrola dostępu na wszystkich chronionych podstronach,
- odporność formularzy na typowe błędne dane,
- komunikaty bez ujawniania informacji technicznych,
- ochrona danych wyświetlanych przed XSS,
- uporządkowany, niepowielany kod logowania i uprawnień,

- czytelne i estetyczne formularze oraz komunikaty,
- poprawna reakcja na celowo błędne dane wejściowe.

Ocena 6 - celujący

- wszystko na ocenę 5,
- kilka poprawnie wykonanych rozszerzeń, np. reset hasła, potwierdzanie e-maila, tokeny CSRF, limit prób logowania, blokowanie konta, rozbudowane uprawnienia, bezpieczne zarządzanie sesją lub zmiana hasła po podaniu starego.

Etap 3. Panel administracyjny

TERMIN	6 listopada 2026 r.
OCENIE PODLEGA	kompletność i jakość panelu służącego do zarządzania systemem

Ocena 1 - niedostateczny

- panel administratora nie działa,
- nie można wykonywać podstawowych operacji na danych,
- poważne błędy uniemożliwiają używanie panelu,
- zwykły użytkownik ma dostęp do funkcji administratora.

Ocena 2 - dopuszczający

- działające logowanie administratora,
- podstawowy panel administracyjny,
- wyświetlanie danych,
- dodawanie przynajmniej części danych, np. kategorii lub usług,
- rozpoczęta implementacja CRUD.

Dopuszczalne są brakujące moduły i niepełna edycja danych.

Ocena 3 - dostateczny

- wszystko na ocenę 2,
- CRUD kategorii i usług,
- zarządzanie pracownikami,
- cena i czas trwania usługi,
- przypisanie usługi do kategorii,
- przeglądanie rezerwacji,
- podstawowa walidacja formularzy,
- dostęp ograniczony do administratora.

Ocena 4 - dobry

- wszystko na ocenę 3,
- przypisywanie wielu usług pracownikowi,
- określanie dostępności pracownika,
- aktywowanie lub dezaktywowanie usług i pracowników,
- zmiana statusu rezerwacji,
- filtrowanie według daty, statusu i pracownika,
- dashboard z podstawowymi informacjami,
- czytelna nawigacja,
- potwierdzanie operacji takich jak usunięcie.

Ocena 5 - bardzo dobry

- wszystko na ocenę 4,
- poprawna walidacja najważniejszych formularzy i obsługa błędów,
- ochrona operacji przed nieuprawnionym użytkownikiem,
- wyszukiwanie lub paginacja większych zbiorów,
- estetyczne i czytelne tabele oraz formularze,

- dashboard z rzeczywistymi danymi,
- bezpieczna obsługa zależności, np. usługi z rezerwacjami,
- uporządkowany kod bez zbędnego powtarzania.

Ocena 6 - celujący

- wszystko na ocenę 5,
- kilka znaczących rozszerzeń, np. wykresy i statystyki, kalendarz administratora, AJAX, zaawansowane filtrowanie, historia zmian, log administratora, eksport danych, urlopy i dni wolne lub rozbudowane uprawnienia pracowników.

Etap 4. Część klienta i system rezerwacji

TERMIN	20 listopada 2026 r.
OCENIE PODLEGA	poprawność procesu rezerwacji i wykrywania konfliktów terminów - najważniejsza logika projektu

Ocena 1 - niedostateczny

- klient nie może poprawnie utworzyć rezerwacji,
- rezerwacja nie jest zapisywana w bazie,
- błędy systemu rezerwacji uniemożliwiają jego używanie.

Ocena 2 - dopuszczający

- wyświetlanie oferty usług,
- wybór usługi i pracownika,
- formularz wyboru daty lub godziny,
- zapis rezerwacji w bazie.

Dopuszczalne są poważniejsze braki w sprawdzaniu dostępności.

Ocena 3 - dostateczny

- wszystko na ocenę 2,
- po wyborze usługi wyświetlani są odpowiedni pracownicy,
- wybór daty i godziny,
- powiązanie rezerwacji z zalogowanym klientem,
- lista własnych rezerwacji klienta,
- informacje o usłudze, pracowniku, terminie, cenie i statusie,
- brak dostępu klienta do rezerwacji innych osób,
- podstawowe sprawdzanie zajętych terminów.

Ocena 4 - dobry

- wszystko na ocenę 3,
- uwzględnianie godzin pracy,
- zakaz rezerwacji w przeszłości i poza godzinami pracy,
- zakaz rezerwacji zajętego terminu,
- anulowanie własnej przyszłej rezerwacji,
- działające statusy,
- panel pracownika z jego wizytami,
- zmiana statusu obsługiwanej wizyty.

Ocena 5 - bardzo dobry

- wszystko na ocenę 4,
- uwzględnianie czasu trwania usług,
- wykrywanie wszystkich nakładających się przedziałów, także częściowo,
- sprawdzenie konfliktu na serwerze bezpośrednio przed zapisem,
- brak możliwości ominięcia kontroli przez zmianę żądania,
- pracownik nie zarządza rezerwacjami innego pracownika,
- klient nie anuluje cudzej rezerwacji,
- poprawna obsługa równoczesnej próby zajęcia tego samego terminu,

- czytelny i wygodny wybór terminu,
- czytelny komunikat przy błędnej próbie zamiast błędu aplikacji.

Wymagany przykład konfliktów: istniejąca rezerwacja 10:00-11:00 blokuje 9:30-10:30, 10:00-11:00, 10:15-10:45, 10:30-11:30 i 9:30-11:30. Dopuszczalne są terminy kończące się dokładnie o 10:00 lub zaczynające dokładnie o 11:00, jeśli spełniają pozostałe warunki.

Ocena 6 - celujący

- wszystko na ocenę 5,
- kilka znaczących rozszerzeń, np. graficzny kalendarz, AJAX, automatyczne dostępne godziny, różne godziny pracy, urlopy i dni wolne, przerwy, e-maile, przypomnienia, przełożenie wizyty, historia statusów lub szczególnie wygodny terminarz.

Etap 5. Gotowy projekt i prezentacja

TERMIN	27 listopada 2026 r.
OCENIE PODLEGA	kompletna aplikacja, bezpieczeństwo, wygląd, kod, Git oraz znajomość projektu

Ocena 1 - niedostateczny

- aplikacji nie można uruchomić,
- podstawowe funkcje nie działają,
- projekt jest nieukończony w stopniu uniemożliwiającym prezentację,
- uczniowie nie potrafią wyjaśnić działania projektu.

Ocena 2 - dopuszczający

- możliwość uruchomienia aplikacji,
- podstawowe logowanie,
- podstawowy panel administratora,
- utworzenie podstawowej rezerwacji,
- zapis danych w bazie.

Projekt może posiadać liczne braki i błędy.

Ocena 3 - dostateczny

- wszystko na ocenę 2,
- rejestracja, logowanie i role,
- oferta usług,
- panel administratora,
- zarządzanie usługami i pracownikami,
- rezerwacja, panel klienta i panel pracownika,
- baza z poprawnymi podstawowymi relacjami.

Dopuszczalne są drobne błędy, które nie uniemożliwiają używania podstawowych funkcji.

Ocena 4 - dobry

- wszystko na ocenę 3,
- wszystkie obowiązkowe elementy projektu,
- poprawny system dostępności i rezerwacji,
- poprawne uprawnienia, podstawowe zabezpieczenia i walidacja,
- czytelny, spójny interfejs i responsywny podstawowy widoki,
- uporządkowany kod i poprawna baza,
- regularna praca z Git,
- aktywny udział obu osób.

Ocena 5 - bardzo dobry

- wszystko na ocenę 4,
- kompletna i stabilna aplikacja bez istotnych błędów w typowym użyciu,
- poprawna reakcja na nieprawidłowe działania użytkownika,
- prepared statements lub ORM i prawidłowo zabezpieczone hasła,
- ochrona danych innych użytkowników,
- estetyczny, responsywny interfejs,

- czytelna struktura kodu bez dużego powielania,
- kompletna instrukcja uruchomienia w README,
- brak sekretów w repozytorium,
- rzeczywista i regularna praca obu osób,
- umiejętność wyjaśnienia kodu i rozwiązań.

Ocena 6 - celujący

- wszystko na ocenę 5,
- projekt wyraźnie wykracza poza podstawy funkcjonalnością i jakością,
- kilka znaczących rozszerzeń, np. zaawansowany kalendarz, AJAX, powiadomienia, e-maile, reset hasła, statystyki, eksport, PDF, historia operacji, urlopy, dodatkowe zabezpieczenia, REST API lub testy automatyczne.

Ocena 6 nie jest przyznawana wyłącznie za framework, bibliotekę, gotowy szablon albo dużą liczbę linii kodu.

14. Zasady oceny indywidualnej i obrony

Ocena za dany etap może być różna dla obu członków zespołu. Repozytorium GitHub jest jednym ze źródeł informacji o podziale pracy.

- autorzy commitów, regularność i zakres zmian,
- gałęzie projektu i historia scalania,
- kod wykonany przez poszczególnych uczniów,
- aktywność podczas zajęć,
- znajomość całego projektu, nie tylko własnego fragmentu.

14.1. Pytania i zadania podczas sprawdzania

- wyjaśnienie wskazanego fragmentu kodu,
- wskazanie miejsca realizującego funkcję,
- wyjaśnienie zapytania SQL i relacji bazy,
- opis mechanizmu logowania lub algorytmu dostępności,
- niewielka modyfikacja kodu,
- znalezienie i poprawienie prostego błędu.

Brak znajomości kodu może skutkować obniżeniem indywidualnej oceny. Uczeń, który nie potrafi wyjaśnić kodu znajdującego się w projekcie, może otrzymać ocenę niższą od partnera.

14.2. Prezentacja końcowa

Nauczyciel może poprosić o rejestrację użytkownika, wykonanie rezerwacji, próbę zajęcia konfliktowego terminu, logowanie jako administrator i pracownik, zmianę usługi lub statusu, pokazanie historii Git oraz wyjaśnienie wybranego fragmentu kodu.

15. Frameworki, biblioteki, szablony i AI

15.1. Frameworki i biblioteki

Są dozwolone, jeżeli uczeń potrafi wyjaśnić sposób ich wykorzystania. Laravel, Bootstrap, Tailwind CSS, jQuery, FullCalendar ani inna biblioteka same w sobie nie podnoszą oceny.

15.2. Gotowe szablony

Można korzystać z gotowych szablonów graficznych. Oceniana jest aplikacja wykonana przez uczniów, a nie wygląd zakupionego lub pobranego szablonu.

15.3. Kod przygotowany z pomocą AI

Narzędzia wspomagające programowanie nie zwalniają z rozumienia kodu. Uczeń musi wiedzieć, za co kod odpowiada, potrafić go wyjaśnić, znaleźć błąd i wykonać niewielką modyfikację. Kod, którego autorzy nie potrafią wyjaśnić, może nie zostać uznany za prawidłowo wykonany element projektu.

15.4. Funkcje dodatkowe

Są uwzględniane tylko wtedy, gdy rzeczywiście działają, są zintegrowane z projektem, mają uzasadnienie i uczniowie potrafią wyjaśnić ich działanie. Niedokończone dodatki nie zastępują funkcji obowiązkowych.

16. Lista kontrolna przed prezentacją

- [] Repozytorium jest dostępne, aktualne i zawiera pracę obu osób.
- [] README zawiera instrukcję uruchomienia i testowe konta.

- [] database.sql odtwarza kompletną bazę, a ERD odpowiada implementacji.
- [] Działają rejestracja, logowanie, wylogowanie, sesje i trzy role.
- [] Działają oferta, panele klienta, pracownika i administratora.
- [] Proces rezerwacji zachowuje kolejność: usługa - pracownik - data - godzina - potwierdzenie.
- [] System uwzględnia godziny pracy, czas usługi i wszystkie konflikty terminów.
- [] Kontrole bezpieczeństwa działają po stronie serwera.
- [] Aplikacja jest responsywna i pokazuje czytelne komunikaty.
- [] Obie osoby potrafią uruchomić projekt, wyjaśnić kod i wykonać drobną zmianę.

17. Najważniejsza zasada projektu

Projekt nie jest konkursem na liczbę linii kodu ani sam wygląd strony. Najważniejsze są:

1. poprawnie zaprojektowana baza danych,
2. poprawna logika aplikacji,
3. bezpieczeństwo,
4. samodzielność,
5. współpraca dwóch osób,
6. systematyczna praca z Git i GitHub,
7. umiejętność wyjaśnienia własnego kodu.